

Project Proposal: "Funcle" (Function Wordle)

System Specification & Architecture Document

1. Executive Summary & Objective

The objective of this project is to develop a web-based, turn-based mathematical deduction puzzle called **"Funcle" (Function Wordle)**. The game adapts the highly successful structural mechanics of *Wordle* (6 constrained attempts, precise feedback, sleek minimalist UI) and shifts the problem space from linguistics to algebraic deduction.

The user plays the role of a "Function Detective," attempting to isolate a secret polynomial function $f(x)$ with integer coefficients within 6 moves using point evaluations (`val x`) and local behavior queries (`is_inc x`).

This project serves as a collaborative development bridge:

- **Mathematical Core:** Designed and verified by the mathematician (your son).
- **Software Architecture:** Engineered, structured, and managed by the IT expert (you).

2. Core Game Mechanics & Example Walkthrough

2.1 The Underlying Rules

1. **The Hidden Target:** The system instantiates a secret polynomial function $f(x) = a_nx^n + a_{n-1}x^{n-1} + \dots + a_1x + a_0$, where all coefficients $a_i \in \mathbb{Z}$ (integers).
2. **The Domain:** $\mathbb{R}$ (All Real Numbers).
3. **The Constraints:** The player has a maximum of **6 turns** total across all query types.
4. **Valid Commands:**
 - `val x`: Returns the exact scalar value of $f(x)$.
 - `is_inc x`: Evaluates the first derivative $f'(x)$. Returns **Increasing** if $f'(x) > 0$, **Decreasing** if $f'(x) < 0$, or a edge-case value if $f'(x) = 0$.
 - `target [expression]`: Compares the user's parsed algebraic input string against the secret function. Correct submission results in a win; incorrect counts as a spent turn.

2.2 Deep-Dive Gameplay Simulation

To ground the logic, here is the exact trace of a game session where the hidden function is $f(x) = x^2 - 4$:

- **Turn 1: Coordinate Discovery**
 - *Player Input:* `val 0`
 - *Backend Evaluation:* $f(0) = 0^2 - 4 = -4$
 - *UI Output:* `[val 0] -> -4` (Rendered as a stable data row)
 - *Deduction:* The y-intercept is located at $(0, -4)$.
- **Turn 2: Second Point Evaluation**

- **Turn 3: Vector/Slope Verification**

- **Turn 4: Symmetry Verification**

- **Turn 5: Final Accusation**

- *Player Input:* `target x^2 - 4`
- *Backend Evaluation:* String or symbolic match confirms structural equivalence.
- *UI Output:* `Success! Match Found in 5 Turns.` (Triggers Win Screen state).

The UI will adopt Wordle's signature aesthetics: a vertical layout optimized for mobile screens, high-contrast dark background, clear grid rows, and structured input selectors instead of raw keyboard text entry to eliminate syntax errors.

FUNCLE		
[?] [Stats] [🌀]		
R1	 val 0	-4
R2	 is_inc 1	Increasing
R3	 [Current Action Line]	
R4		

R5

+-----+

R6

+-----+

+-----+

Select Action:Input x:Execution:

+-----+ +-----+ +-----+

valv

2

SUBMIT CLUE

+-----+ +-----+ +-----+

[Remaining Guess Bullets: 4 / 6]

+-----+

UI Design Highlights for Frontend Engineering (Vue.js):

- **The History Grid:** 6 rows containing component cards. When an action is submitted, it appends to the grid using colored markers (e.g., Green for numerical coordinates, Yellow for directional limits).
- **The Command Matrix:** Rather than forcing users to type syntax, dropdowns control the command selection (`val`, `is_inc`, `target`), preventing input errors.
- **State Control:** Input components are disabled automatically when the turn counter reaches 0 or when the win condition evaluates to true.

4. Technical Architecture Stack (IT Perspective)

- **Frontend Engine:** Vue.js (Single Page Application architecture). It handles reactive states smoothly for the 6-row tracking grid.
- **Styling Architecture:** Modern CSS grid and flex structures mapped inside explicit layouts, utilizing a dark-mode palette (`#121213` canvas, `#538d4e` success green, `#b59f3b` directional yellow).
- **State Management:** Reactive state tracking remaining turns, input histories, and parsed string tokens.
- **Mathematical Engine:** JavaScript native parsing or a lightweight utility package (like `mathjs`) to safely parse string targets (`target x^2 - 4`) and compute analytical derivatives dynamically.

5. Outstanding Questions for Math Validation

To ensure a robust system architecture, please have your son clarify the following technical math rules before coding begins:

1. Polynomial Degree Boundary (n):

- What is the maximum degree allowed for the secret function? Is it limited to quadratics ($n = 2$, e.g., $x^2 + 2x + 1$), cubics ($n = 3$), or can it scale higher?
- *Why this matters to IT:* This determines how we generate functions randomly without breaking game balance.

2. Coefficient Constraints (a_i):

- What is the acceptable range for the integer coefficients? (e.g., integers between -10 and $+10$). Can the leading coefficient a_n be negative?
- *Why this matters to IT:* Prevents the generator from producing unplayable equations like $f(x) = 87x^2 - 53x + 99$.

3. Derivative Evaluation at Critical Points ($f'(x) = 0$):

- If the user executes `is_inc x` exactly at a peak, valley, or inflection point where the derivative evaluates to exactly 0 (e.g., at $x = 0$ for $f(x) = x^2$), what should the engine return?
- *Options:* Should it output `Stationary`, `Zero`, or display an explicit error/hint?

4. Domain Constraints for Input x :

- Is the player limited to inputting integers for x when using `val x` and `is_inc x`, or can they enter decimals/fractions (e.g., `val 1.5`)?
- *Why this matters to IT:* Dictates whether our input validation field needs to accept floats or strictly integers.

5. String Target Equivalence Rule:

- How strict is the math matching algorithm? If the secret is $x^2 - 4$, and the player inputs `target -4 + x^2` or `target (x-2)(x+2)`, does the game evaluate this as correct?
 - *Why this matters to IT:* If order does not matter, the backend must use a symbolic math parser to simplify the user's expression before checking equality, rather than checking standard text strings.
-

6. Next Steps

1. **Review & Confirm:** Math student to review definitions and provide answers to Section 5.
2. **Database / State Modeling:** Formulate the JSON payload template for tracking state history.
3. **Prototyping:** Begin coding the core math engine evaluation loop.